



## 机器视觉实验 4

姓名：冯怡然

学号：2411434

班级：人工智能特色班

日期：2026 年 4 月 13 日

南开大学 · 机器视觉实验

# 1 题目二：迭代的最优阈值选择

## 1.1 实验目的

1. 掌握“迭代最优阈值选择”算法的数学原理与实现过程。
2. 学习如何通过 MATLAB 编程实现自动化阈值选取，并分析算法的收敛性。

## 1.2 实验原理

迭代式阈值选择算法是一种基于图像统计特性的自动寻找分割阈值的方法。其核心思想是将图像像素分为目标和背景两部分，并通过不断更新阈值使得该阈值位于两部分平均灰度的中心。

其具体数学推导如下：

1. **初始化阈值**：选择一个初始阈值  $T_0$ 。通常取图像灰度范围的中点：

$$T_0 = \frac{Z_{max} + Z_{min}}{2}$$

其中  $Z_{max}$  和  $Z_{min}$  分别是图像中像素的最大和最小灰度值。

2. **区域划分**：利用当前阈值  $T_k$  将图像划分为两组像素集合：

- $G_1$ ：所有灰度值  $> T_k$  的像素（通常视为目标）。
- $G_2$ ：所有灰度值  $\leq T_k$  的像素（通常视为背景）。

3. **计算均值**：分别计算这两个区域的平均灰度值  $\mu_1$  和  $\mu_2$ ：

$$\mu_1 = \frac{\sum_{(i,j) \in G_1} f(i,j)}{N_{G_1}}, \quad \mu_2 = \frac{\sum_{(i,j) \in G_2} f(i,j)}{N_{G_2}}$$

4. **更新阈值**：计算新的阈值  $T_{k+1}$ ：

$$T_{k+1} = \frac{\mu_1 + \mu_2}{2}$$

5. **收敛判断**：重复步骤 2 至 4，直到相邻两次迭代的阈值差小于预设的微小值  $\Delta T$ （如 0.5 或 0）：

$$|T_{k+1} - T_k| < \Delta T$$

## 1.3 实验步骤

1. **图像预处理**：使用 `imread` 函数读取彩色测试图像，并通过 `rgb2gray` 函数将其转换为灰度图像，为后续的阈值处理做准备。
2. **迭代循环计算**：建立一个 `while` 循环。在循环内部：
  - 运用逻辑索引找出大于和小于等于当前阈值的像素点。
  - 使用 `mean` 函数计算两类像素的平均灰度。
  - 记录每次迭代的  $T$  值，存入数组中以便后续绘图。
  - 判断新旧阈值的绝对差。若满足停止条件，则跳出循环。
3. **收敛性分析**：利用记录的阈值变化数据，使用 `plot` 函数绘制阈值随迭代次数变化的曲线图，观察算法的稳定过程。
4. **二值化输出**：根据最终得到的最佳阈值  $T$ ，对原灰度图像进行逻辑判断 ( $img > T$ )，生成二值化结果图像。

## 1.4 程序代码 (MATLAB)

### 1.4.1 main.m

```
1 clc;clear all;close all;
2 img=imread('ratatouille.png');
3 % 彩色图转为灰度图
4 img=rgb2gray(img);
5 %迭代的最优阈值选择, 需编程实现
6 %鼓励尝试同时输出阈值t的变化曲线图
7 %可以稍微修改这个源程序, 比如增加下面这个函数的返回值等
8 OptimalThreshold_result = MyOptimalThreshold(img);
9 figure;
10 imshow(OptimalThreshold_result);
11 title('OptimalThresholdResult');
```

Listing 1: main.m

### 1.4.2 MyOptimalThreshold.m

```
1 function OptimalThreshold_result=MyOptimalThreshold(img)
2 %完善迭代最优阈值选择的计算过程
3 img_double=double(img);
4 T=(max(img_double(:))+min(img_double(:)))/2;
5 Thistory=[];
6 deltaT=0.3;
7 while true
8     Thistory=[Thistory,T];
9     g1=img_double(img_double>T);
10    g2=img_double(img_double<=T);
11    mu1=mean(g1);
12    mu2=mean(g2);
13    newT=(mu1+mu2)/2;
14    if abs(newT-T)<deltaT
15        break;
16    end
17    T=newT;
18 end
19 figure;
20 plot(Thistory, 'o-', 'LineWidth', 1.5);
21 grid on; title('阈值 T 的迭代变化曲线');
22 xlabel('迭代次数'); ylabel('阈值大小');
23 %返回原图像经过迭代最优阈值选择处理后的图像OptimalThreshold_result
24 OptimalThreshold_result = img_double > T;
25 end
```

Listing 2: MyOptimalThreshold.m

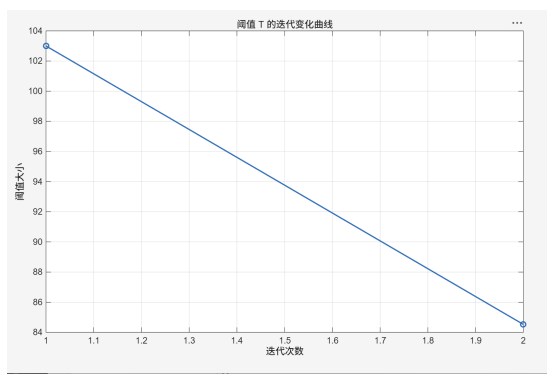
## 1.5 实验结果显示

可以看到，随着  $\Delta T$  变小，迭代次数增多，图像分割越精确



图 1: 原图片

$\Delta T=15$ ，共迭代 2 次， $T$  停留在 84 左右。可以看见 Remy 的一点胡子和一点双眼皮，Chef Gusteau 的阴影半边脸没有被提取出来



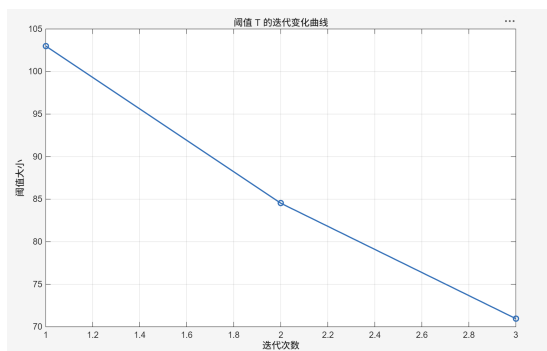
(a)



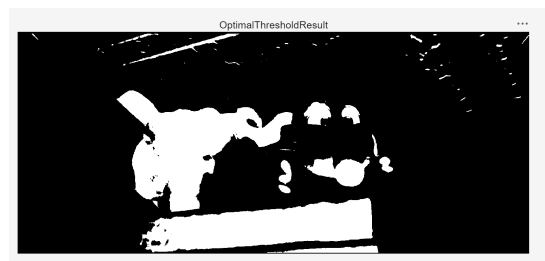
(b)

图 2:  $\Delta T=10$

$\Delta T=10$ ，共迭代 3 次， $T$  停留在 71 上下。可以看见 Remy 的 2 根胡子和潦草双眼皮，Chef Gusteau 的脸基本融为一体



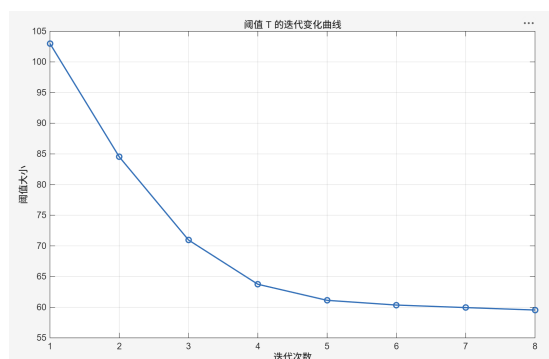
(a)



(b)

图 3:  $\Delta T=10$

$\Delta T=0.3$ , 迭代 8 次,  $T$  停留在 60 上下。这时候 Remy 的双眼皮和眼睛一起被提取出来, 三根胡子也很清楚, Chef Gustau 的整张脸都被提取出来, 后面的柱子光影细节也更加突出



(a)



(b)

图 4:  $\Delta T=10$

## 1.6 实验分析总结

- 算法有效性:** 实验证明迭代法能有效自动确定图像分割阈值。算法通过不断优化目标与背景的均值中心, 使最终阈值  $T$  能够较好地适应图像的灰度分布, 实现自动二值化。
- 参数灵敏度:** 迭代停止条件  $\Delta T$  对结果影响显著。较大的  $\Delta T$  会导致迭代提前终止, 阈值偏离最优值, 造成细节丢失 (如胡子和面部阴影); 而减小  $\Delta T$  能增加迭代次数, 使阈值向最优解收敛, 显著提升细节的提取精度。
- 收敛特性:** 从阈值变化曲线可见, 算法具有良好的收敛性。初始迭代时阈值变动剧烈, 随后逐渐趋于平缓并锁定在最优值附近, 这体现了该算法在图像自动分割中的稳定性和可靠性。

## 2 题目三——大津法 OSTU 阈值检测 (MATLAB)

### 2.1 实验目的

1. 深入理解 OTSU (大津法) 算法的基本原理, 即通过最大化类间方差来寻找最优阈值。
2. 能够使用 MATLAB 编写高效的 OTSU 算法函数, 并对比其与迭代阈值法在分割效果上的差异。

### 2.2 实验原理

Otsu 算法是一种统计性阈值选择方法。其基本思想是: 将图像像素按灰度级分为  $C_0$  (背景) 和  $C_1$  (前景) 两类, 通过搜索使这两类之间方差 (Between-class Variance) 达到最大时的灰度值作为最优阈值。

设图像灰度级为  $\{0, 1, \dots, L-1\}$  (通常  $L = 256$ ), 其具体的数学推导如下:

1. **概率分布**: 计算每个灰度级  $i$  出现的概率  $p_i$ :

$$p_i = \frac{n_i}{N}$$

其中  $n_i$  为灰度级  $i$  的像素数,  $N$  为总像素数。

2. **类累积概率**: 对于选定的阈值  $t$ , 背景  $C_0$  和前景  $C_1$  的占比分别为:

$$\omega_0(t) = \sum_{i=0}^t p_i, \quad \omega_1(t) = 1 - \omega_0(t)$$

3. **类平均灰度**: 两类的平均灰度  $\mu_0$  和  $\mu_1$  以及整幅图的均值  $\mu$  为:

$$\mu_0(t) = \frac{\sum_{i=0}^t i \cdot p_i}{\omega_0(t)}, \quad \mu = \sum_{i=0}^{L-1} i \cdot p_i$$

根据均值定义, 有  $\mu = \omega_0 \mu_0 + \omega_1 \mu_1$ 。

4. **类间方差**: 目标函数为最大化类间方差  $\sigma_B^2(t)$ :

$$\sigma_B^2(t) = \omega_0(t) \cdot \omega_1(t) \cdot [\mu_0(t) - \mu_1(t)]^2$$

5. **确定阈值**: 遍历所有灰度级,  $t^*$  即使  $\sigma_B^2(t)$  最大的值。

### 2.3 实验步骤

1. **直方图统计**: 使用 `imhist` 获取图像各灰度级的分布, 并归一化得到概率分布数组  $p$ 。
2. **预计算全局特性**: 生成  $0 \sim 255$  的灰度向量, 计算全图的平均灰度  $\mu$ 。
3. **遍历计算方差**:
  - 建立一个从 1 到 256 的循环。
  - 计算当前阈值下的背景占比  $\omega_0$ 。若  $\omega_0$  为 0 或 1, 则跳过该循环以避免分母为 0。
  - 计算背景均值  $\mu_0$  和前景均值  $\mu_1$ 。

- 计算并存储当前的类间方差  $\sigma_B^2$ 。
4. **寻找极值点**: 利用 `max` 函数找到方差数组中最大值的位置, 将其索引转换为灰度阈值  $T = \text{max\_idx} - 1$ 。
  5. **结果生成**: 根据  $T$  对原图进行逻辑判定, 生成二值化结果图并显示。

## 2.4 程序代码 (Matlab)

### 2.4.1 main.m

```

1 clc;clear all;close all;
2 img=imread('ratatouille.png');
3 % 彩色图转为灰度图
4 img=rgb2gray(img);
5 %OTSU阈值检测, 需编程实现
6 OTSU_result = MyOTSU(img);
7 figure;
8 imshow(OTSU_result);
9 title('OTSUResult');
```

Listing 3: main.m

### 2.4.2 MyOSTU.m

```

1 function OTSU_result=MyOTSU(img)
2 %完善OTSU阈值检测的计算过程
3 counts=imhist(img);
4 p=counts/sum(counts);
5 graylevels=(0:255);
6 mu=sum(graylevels.*p);
7 sigma2=zeros(256,1);
8 for t=1:256
9     w0=sum(p(1:t));
10    w1=1-w0;
11    if w0==0 || w1==0
12        continue;
13    end
14    mu0=sum(graylevels(1:t).*p(1:t))/w0;
15    mu1=(mu-w0*mu0)/w1;
16    sigma2(t)=w0*w1*(mu0-mu1)^2;
17 end
18 [~, max_idx] = max(sigma2);
19 best_T = max_idx - 1;
20 %返回原图像经过OTSU最优阈值化后的二值图OTSU_result
21 OTSU_result=img>best_T;
22 end
```

Listing 4: MyOSTU.m

## 2.5 实验结果显示

可以看到，OTSU 算法的结果同迭代法中， $\Delta T$  较小的结果类似，整体来说非常准确。



图 5: OTSU

## 2.6 实验分析总结

1. **算法准确性：**实验结果显示，OTSU 算法能够自动识别出 Remy 的胡子细节、Chef Gustau 的面部轮廓以及背景的光影。相比于手动设置阈值，OTSU 具有更强的自适应性。
2. **对比结论：**通过观察发现，OTSU 的分割效果与迭代法在  $\Delta T$  较小（即收敛精度高）时的结果高度相似。这说明在单峰或类双峰的直方图分布下，OTSU 能非常精准地定位到背景与目标的分界线。
3. **性能分析：**OTSU 算法只需要对灰度分布进行一次遍历，计算开销恒定且效率极高，不需要像迭代法那样可能面临多次循环。它是一种在工程实践中平衡了速度与精度的优秀全局阈值算法。